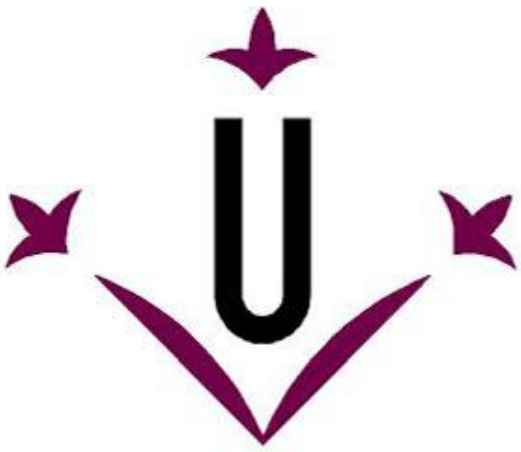

Universitat de Lleida

High Performance Computing
Convolution Operation OpenMP-MPI Practice

Students:
Joel Castillo Estallo
David Carreras Castañ

Date: 17 May 2026

Main decisions taken to parallelize the problem

The goal of this delivery was to extend the previous OpenMP solution towards a hybrid MPI+OpenMP implementation of the convolution program. The starting point of the work was the serial convolution application provided in the assignment, used to obtain serial times, together with the OpenMP version developed in the previous delivery. In this sense, the hybrid solution was not designed from scratch: it was built as a natural evolution of the OpenMP approach, preserving the row-based shared-memory parallelism inside each process and adding a distributed-memory decomposition across nodes.

As in the previous delivery, the dominant computational phase of the application is the convolution itself. In the original sequential/OpenMP structure, the convolution is applied independently to the three colour channels, and the most expensive work is concentrated in the nested loops that compute each output pixel from its surrounding neighbourhood. For this reason, the OpenMP decision adopted in the previous report was preserved: the local convolution performed by each process continues to use a row-level OpenMP parallel loop, since different rows of the output image can be computed independently and only read shared input data and kernel coefficients. This preserves the main strength of the OpenMP version developed in the first report.

The main new design decision of the hybrid version was the distributed-memory decomposition. We decided to implement a static mapping of tasks, where the work is divided into a fixed set of fragments assigned at the beginning of the execution. The image was decomposed by contiguous row blocks, and each MPI process received a fixed block of useful rows together with the additional halo rows required by the convolution kernel. This allocation is computed once at the beginning of the execution and is not modified later, so the chosen distributed-memory strategy clearly corresponds to a static mapping of tasks.

The choice of a row-based decomposition was motivated by several reasons. First, it is consistent with the OpenMP strategy already used in the previous delivery, where the work inside the convolution kernel was also parallelized by rows. Second, row blocks are easy to define and gather again at the end of the execution. Third, since the convolution kernel requires neighbouring pixels, a row-based decomposition allows the halo exchange to be expressed in a direct way by extending each block with several extra rows above and below the useful region. This made the implementation easier to reason about than alternatives such as quadrants or more irregular partitions. At the same time, the strategy remains close to the original program structure, which was one of the strengths of the previous OpenMP implementation.

During development, an important refinement was introduced in relation to the halo management. A first straightforward approach is to distribute each local block including halo rows and then apply the convolution over the whole received block. However, this causes unnecessary work, because halo rows

are only required as read-only context for neighbouring useful rows and should not be fully recomputed as if they were part of the final local result. Therefore, the final version adapts the local convolution routine so that each process only computes its useful rows, while the halo rows are kept only for neighbourhood access. This reduces useless computation and is especially important for large kernels such as 99x99, where the halo becomes significant.

From the implementation point of view, the final hybrid structure follows a master-centred design. Rank 0 reads the full image and the kernel, broadcasts the global metadata and kernel coefficients, computes the static decomposition, distributes the corresponding row blocks with halo to the remaining ranks, gathers the useful rows produced by each process, reconstructs the final image and writes the output file. The worker ranks are therefore devoted mainly to the computational phase, as it is the most cost efficient task. This design was selected because it is simpler and more robust than a fully distributed I/O design, and it was especially convenient during debugging and validation of the MPI communications. Nevertheless, this also introduces extra coordination work on rank 0, which later becomes relevant in the performance discussion.

Finally, the timing methodology was adapted to the hybrid environment. In the OpenMP delivery, the timing code had already been changed to use omp_get_wtime(). In the hybrid delivery, the statement recommends the use of MPI_Wtime(), so the final implementation measures the main phases of the MPI+OpenMP execution using this routine: image reading, kernel reading, row-block distribution, local convolution, gathering of useful rows, writing of the result image, and total execution time. This preserves the phase-oriented analysis style of the previous report while adapting it correctly to the distributed-memory execution model.

The source code can be found in the github repository:
https://github.com/JoelCastEst/HPC-OMP-Project

Alternatives considered

Different alternatives were considered before fixing the final solution. First, a **dynamic mapping of tasks** was considered instead of a static one. This option was not chosen because the convolution workload is regular enough to be handled efficiently with a fixed block decomposition, while a dynamic scheme would have increased implementation complexity without a clear expected benefit for the benchmark cases considered.

Second, a **distributed-memory decomposition based on RGB channels** was also considered, assigning one complete colour channel to each MPI worker. This approach is simpler in terms of communication because it avoids halo management, and it was useful as a conceptual reference during debugging. However, it was not selected as the final reported solution because the row-based static mapping provided a more natural continuation of the OpenMP row-parallel design developed in the previous delivery.

Finally, a more **distributed I/O strategy** was also possible, in which each MPI process would read and/or write its corresponding image fragment instead of concentrating the full image reading and final output writing on rank 0. Although this could potentially reduce part of the master-side overhead, it was not implemented because it would have made the program structure and the validation of the final output considerably more complex. As a result, the final design kept a master-centred I/O scheme together with a static row-based decomposition.

Scalability of the Program

To evaluate the scalability of the hybrid implementation, we measured the execution time of the MPI+OpenMP version for the image and kernel combinations required by the assignment and compared the obtained results with those from the OpenMP implementation developed in the previous delivery. In this second report, scalability must be analysed not only for different problem sizes, but also for different numbers of processes/cores and nodes, since the hybrid version combines distributed-memory parallelism through MPI with shared-memory parallelism through OpenMP.
For the hybrid implementation, we evaluated different MPI/OpenMP configurations in order to study the effect of scaling across nodes. In our case, the most relevant configurations were based on keeping 4 OpenMP threads per MPI process and varying the number of MPI processes.
The standard formulas used were:

$$Speedup = \frac{T_{ref}}{T_{hybrid}} \qquad Efficiency = \frac{Speedup}{p}$$

Where $T_{ref}$is the execution time of the previous OMP version, $T_{hybrid}$is the execution time of the Hybrid version, and p is the number of cores used in the hybrid execution. In our executions, p was 8 (2x4) or 16 (4x4) depending on the configuration (MPI Processes x OMP threads per process).
This is done as required by the assignment, to compare the hybrid solution with the previous OMP solution, not the Serial solution, so lower speedups will appear. This is acknowledged, and to compare with the serial execution, T_{ref}, should be the serial time of the execution.
Tables 1 to 5 show the measured execution times together with the corresponding speedup and efficiency values for each image and each kernel. In order to keep the report concise and readable, we chose to preserve the exact values in tables and also provide two summary plots: one overall plot for speedup and one plot for efficiency.

Image Im01	Kernel			
	kernel5x5_Sharpen.txt	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	0.112099	0.641137	2.197561	8.308373
OMP 4 Threads	0.077999	0.352565	1.145758	4.280987
MPI 2 x OMP 4	0.367369	0.471509	0.861126	2.179506
MPI 4 x OMP 4	0.823422	0.915604	1.319237	2.591515
2 x 4 Speedup	0.212	0.748	1.331	1.964
2 x 4 efficiency	0.027	0.093	0.166	0.246
4 x 4 Speedup	0.095	0.385	0.869	1.652
4 x 4 efficiency	0.006	0.024	0.054	0.103

Table 1. Table with arguments to calculate the elapsed times of the Im01.

Image Im02	Kernel			
	kernel5x5_Sharpen.txt	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	0.192128	1.621497	5.636850	22.308050
OMP 4 Threads	0.161151	0.871742	2.917048	11.133542
MPI 2 x OMP 4	0.670408	0.918311	1.935486	5.478338
MPI 4 x OMP 4	1.186121	2.023907	3.118929	5.961890
2 x 4 speedup	0.240	0.949	1.507	2.032
2 x 4 efficiency	0.030	0.119	0.188	0.254
4 x 4 Speedup	0.136	0.431	0.935	1.867
4 x 4 efficiency	0.008	0.027	0.058	0.117

Table 2. Table with arguments to calculate the elapsed times of the Im02.

Image Im03	Kernel			
	kernel5x5_Sharpen.txt	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	0.436407	3.317847	11.481928	46.686213
OMP 4 Threads	0.323869	1.729308	5.867542	22.389592
MPI 2 x OMP 4	1.202094	1.692753	3.543818	10.940957
MPI 4 x OMP 4	1.974338	3.342716	5.222613	12.933911
2 x 4 Speedup	0.269	1.022	1.656	2.046
2 x 4 efficiency	0.034	0.128	0.207	0.256
4 x 4 Speedup	0.164	0.517	1.123	1.731
4 x 4 efficiency	0.010	0.032	0.070	0.108

Table 3. Table with arguments to calculate the elapsed times of the Im03.

Image Im04	Kernel			
	kernel5x5_Sharpen.txt	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	17.955136	162.369510	581.173065	2529.892722
OMP 4 Threads	14.464880	84.165687	289.594108	1139.728952
MPI 2 x OMP 4	47.585708	76.252883	170.539579	543.431546
MPI 4 x OMP 4	84.661016	115.333869	214.577775	577.955941
2 x 4 Speedup	0.304	1.104	1.698	2.097
2 x 4 efficiency	0.038	0.138	0.212	0.262
4 x 4 Speedup	0.171	0.730	1.350	1.972
4 x 4 efficiency	0.011	0.046	0.084	0.123

Table 4. Table with arguments to calculate the elapsed times of the Im04.

Image Im05	Kernel			
	kernel5x5_Sharpen.txt	Kernel25x25_Random2.txt	Kernel49x49_Random2txt	Kernel99x99-Random2.txt
Serial	65.472774	643.877559	2340.317637	10373.402927
OMP 4 Threads	50.160328	341.993654	1145.793739	4499.576473
MPI 2 x OMP 4	188.390142	311.560984	667.924729	2164.431486
MPI 4 X OMP 4	326.198191	444.432260	810.927725	2311.810490
2 x 4 Speedup	0.266	1.098	1.715	2.079
2 x 4 efficiency	0.033	0.137	0.214	0.260
4 x 4 Speedup	0.154	0.770	1.413	1.946
4 x 4 efficiency	0.010	0.048	0.088	0.122

Table 5. Table with arguments to calculate the elapsed times of the Im05.

As said before, we decided to represent the values in a table, as we consider, tables capture the exact information that plots sometimes can't. Even so, we decided to plot two overall graphics, for Speedup [Image 1] and for Efficiency [Image 2]

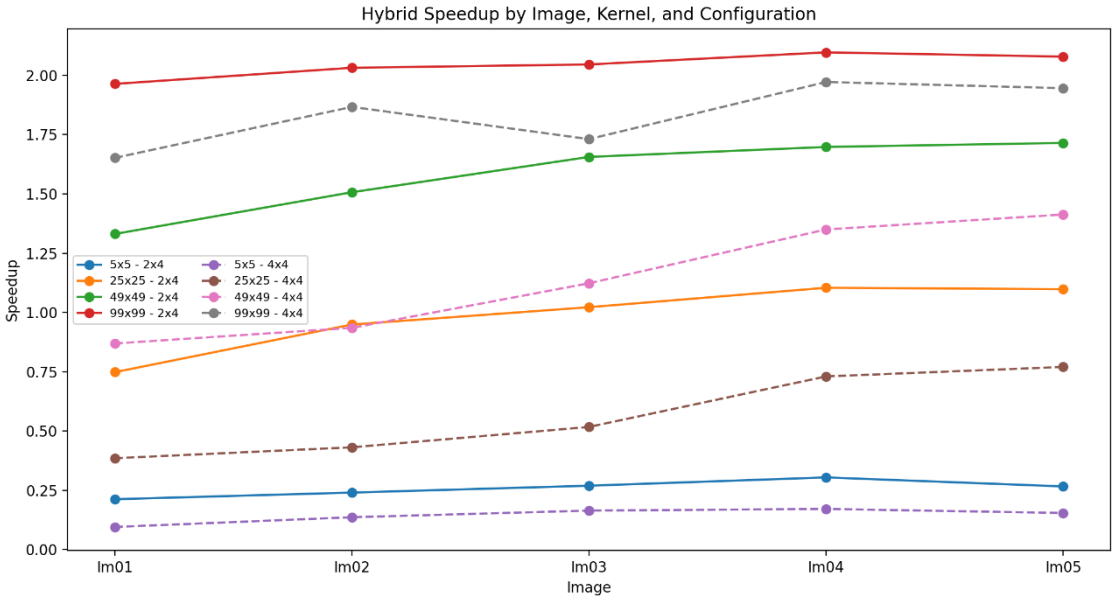

Image 1. Graph for overall Speedup by Image, Kernel and MPI+OMP Count

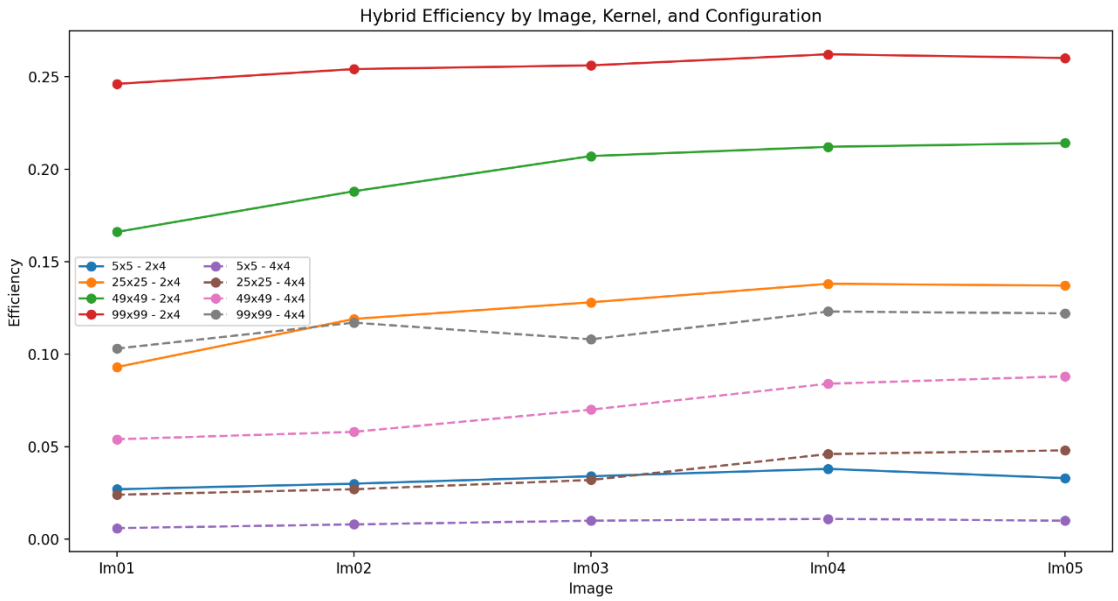

Image 2. Graph for overall Efficiency by Image, Kernel and Thread Count

The obtained results show that the hybrid MPI+OpenMP version does not behave uniformly across all problem sizes. For the smallest kernels, especially 5x5, the hybrid implementation is slower than the OpenMP-only version, which leads to speedup values below 1 and very low efficiency. This indicates that, in small workloads, the additional MPI overhead associated with data distribution, synchronization, gathering and master-side coordination is not compensated by the available parallel work.

As the kernel size increases, the behaviour improves significantly. For the 49x49 and especially the 99x99 kernels, the hybrid version clearly outperforms the OpenMP reference, reaching the best speedup values in the largest images. This confirms that the hybrid strategy becomes more profitable when the computational workload is large enough to amortize the communication and coordination costs introduced by MPI.

Another relevant result is that the 2x4 configuration often performs better than 4x4. Although 4x4 uses more total cores, it also increases the MPI overhead: more processes imply more block boundaries, more halo management, more communications, and more gathering work for the master. Since each process already exploits 4 OpenMP threads efficiently, the additional distributed-memory parallelism of 4x4 does not always compensate for its extra communication cost. This explains why an intermediate configuration such as 2x4 can provide a better global balance between computation and overhead.

Overall, the results indicate that the hybrid solution is advantageous for medium and large workloads, particularly with large kernels, but it is less effective for small cases where the MPI overhead dominates. The figures and tables consistently show that the best hybrid behaviour is obtained when the computational intensity is high enough to justify the distributed-memory execution model.

Discussion of the obtained results

The obtained results can be explained by considering the structure of the convolution algorithm, the selected hybrid parallelization strategy, and the overhead introduced by distributed-memory execution. In the final implementation, the image is decomposed into static row blocks, each MPI process receives its corresponding useful rows together with the halo rows required by the convolution kernel, and the local computation is then parallelized with OpenMP inside each process. Therefore, the final performance depends on two different levels of parallelism: the distribution of work across MPI processes and the shared-memory row-level parallelism inside each process.

As in the previous OpenMP delivery, the computational cost of the problem grows strongly with both image size and kernel size. A larger image increases the total number of pixels to be processed, while a larger kernel increases the number of neighbour accesses and arithmetic operations required to compute each output pixel. For this reason, the hybrid version shows its best behaviour in the largest and most computationally intensive cases, especially for the 49x49 and 99x99 kernels. In these cases, the local convolution phase dominates the execution time and the additional MPI overhead can be amortized more effectively. This is exactly what can be observed in the measured tables, where the largest speedup values appear for the largest images and kernels.

In contrast, the smallest kernel, 5x5, produces poor hybrid performance in all images. The corresponding speedup values are below 1 for both 2x4 and 4x4, which means that the hybrid implementation is slower than the OpenMP reference. This behaviour is expected because, for small workloads, the amount of useful local computation is too small compared with the cost of distributing row blocks, synchronizing processes, handling halo regions, gathering the partial results and reconstructing the final image on rank 0. In other words, for small kernels the MPI overhead dominates and prevents the hybrid strategy from being profitable.

From the point of view of dependences, the selected strategy is favourable. Different output rows can be computed independently, and each MPI process writes only to its own local output buffer. Inside each process, OpenMP follows the same idea, since different threads compute different output rows and only read shared input data and kernel coefficients. The halo rows are required only as read-only context to provide neighbouring values at block boundaries. Therefore, the convolution kernel presents very low true dependences, which

makes row-based decomposition a natural and efficient approach both at MPI level and at OpenMP level. Synchronization is still needed between phases, especially during distribution and gathering, but not inside the main local convolution loop itself.

Memory traffic is another relevant factor in the observed results. The convolution algorithm already performs many accesses to the input image, and this cost increases with kernel size. In the hybrid version, this memory traffic is combined with communication traffic, because halo-extended row blocks must be distributed to the MPI processes and the useful rows must later be gathered again. Even though the final version avoids recomputing halo rows as useful output, halo data still has to be transferred and stored locally. This extra traffic is relatively more expensive when many MPI processes are used or when the useful workload per process is not large enough. As a result, scalability is limited not only by arithmetic cost, but also by the cost of memory accesses and inter-process communication.

A particularly relevant result is that the 2x4 configuration often performs better than 4x4, even though 4x4 uses more total cores. This is one of the most meaningful observations of the experiments. The reason is that increasing the number of MPI processes also increases the distributed-memory overhead. With more MPI processes, the image is divided into more row blocks, which creates more block boundaries, more halo management, more communication, and more work for the master during distribution and gathering. Since each MPI process already uses 4 OpenMP threads efficiently inside a node, the additional MPI-level parallelism of 4x4 does not always compensate for the extra overhead it introduces. For this reason, 2x4 often provides a better compromise between useful computation and communication cost. This effect can be clearly observed in the measured tables and summary graphs.

In the hybrid version, the additional overhead of parallelization is mainly represented by the phases of image reading, row-block distribution, gathering of useful rows, and final output writing. These phases are absent or much less significant in the OpenMP-only version and therefore provide a practical quantification of the extra cost introduced by MPI coordination.

The efficiency values support the same interpretation. Since the hybrid speedup has been computed with respect to the OpenMP version from the previous delivery, rather than with respect to the serial code, the efficiency values are naturally lower than the classical serial-based efficiencies. Even so, they are still useful for comparing hybrid configurations. In general, 2x4 shows higher efficiency than 4x4, which confirms that the extra cores of the larger configuration are not translated into proportional performance gains. This again points to the importance of communication and coordination overhead in the hybrid execution.

Overall, the results show that the hybrid implementation is advantageous only when the computational intensity of the problem is high enough. For medium and large kernels, especially in the largest images, the hybrid solution improves the

OpenMP baseline and benefits from the combined use of MPI across nodes and OpenMP inside each node. However, for small kernels and light workloads, the distributed-memory overhead outweighs the benefits of the hybrid execution. Therefore, the final performance of the hybrid version is determined by the balance between useful local computation and the additional cost introduced by communication, synchronization, halo handling and master-side coordination.

Conclusions

In this delivery, we extended the previous OpenMP implementation of the convolution program towards a hybrid MPI+OpenMP solution based on a static row-wise decomposition with halo regions. The final implementation combines distributed-memory parallelism across nodes through MPI with shared-memory parallelism inside each process through OpenMP.

The obtained results show that the hybrid approach is not beneficial for small workloads, where the communication and coordination overhead dominates, but it becomes advantageous for medium and large kernels, especially in the largest images, where the computational intensity is high enough to amortize the MPI overhead. A particularly relevant result is that the 2x4 configuration often performs better than 4x4, which confirms that increasing the number of MPI processes does not necessarily improve total performance if the additional communication, halo management and master-side coordination costs grow too much. Overall, the hybrid strategy proved to be effective for the most demanding cases, although its scalability is clearly limited by distributed-memory overhead and by the central role of rank 0 in the current design.

Bibliography

1. **Open MPI Project. *MPI_Wtime Manual Page*.**
   Official reference for the MPI timing routine used to measure the execution phases of the hybrid implementation. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Wtime.3.html
2. **MPI Forum. *MPI Documents*.**
   Official reference for MPI standards and distributed-memory communication concepts.
   https://www.mpi-forum.org/docs